

C++ Exercise : 02. C++ 핵심 문법과 확장

1. 이름 공간(Namespace)에 대한 설명으로 올바른 것은? (하)

1. `using namespace std;`를 헤더 파일의 전역 범위에 선언하면 이름 충돌 위험을 완벽히 방지할 수 있다.
2. 이름 없는 이름 공간(Unnamed namespace) 내의 요소는 해당 파일(번역 단위) 내부에서만 유효하다.
3. 인라인 이름 공간(`inline namespace`)은 부모 이름 공간에서 자식 공간의 이름을 직접 참조할 수 없도록 강제한다.
4. 이름 공간은 오직 전역 변수의 관리를 위해서만 사용할 수 있다.

2. 부동소수점을 콘솔에 출력할 때 소수점 이하의 자릿수를 세밀하게 제어하기 위해 사용하는 `iomanip` 라이브러리의 조작자는 무엇인가? (하)

1. `std::setw`
2. `std::setfill`
3. `std::showbase`
4. `std::setprecision`

3. `std::cin`을 사용하여 입력받을 때 공백이 포함된 한 줄의 문자열 전체를 안전하게 입력받기 위한 가장 적절한 방법은? (하)

1. `std::cin >> stringVariable;`
2. `std::getline(std::cin, stringVariable);`
3. `std::cin.get();`
4. `scanf("%s", &stringVariable);`

4. 다음 코드 중 입력 스트림 실패 상태를 초기화하고 버퍼에 남은 잘못된 입력을 비우기 위해 올바른 순서로 작성된 것은? (중)

- (A) `//: 에러 플래그 초기화`
(B) `//: 입력 버퍼 비우기`

1. (A) `cin.ignore()`, (B) `cin.clear()`
2. (A) `cin.clear()`, (B) `cin.ignore()`
3. (A) `cin.flush()`, (B) `cin.sync()`
4. (A) `cin.empty()`, (B) `cin.reset()`

5. 다음 데이터 크기를 나타내는 타입인 `std::size_t`에 대한 설명으로 가장 올바른 것은? (중)

1. 메모리 주소의 차이를 나타내기 위한 부호 있는 정수형이다.
2. 부동소수점의 바이트 크기를 반환하는 연산자이다.
3. 현재 플랫폼에서 객체나 배열의 최대 크기를 표현할 수 있는 부호 없는 정수형이다.

4. 문자열의 실제 길이를 저장하기 위한 가변 길이 타입이다.

6. C++20부터 도입된 UTF-8 전용 문자 인코딩을 처리하기 위한 기본 타입은 무엇인가? (하)

1. char
2. wchar_t
3. char16_t
4. char8_t

7. 부동소수점형 변수의 값을 비교할 때 발생하는 오차 문제를 해결하기 위한 가장 올바른 대처 방법은 무엇인가? (상)

1. == 연산자를 사용하여 두 값이 정확히 일치하는지 확인한다.
2. 값을 정수형으로 강제 캐스팅한 뒤 == 연산자로 비교한다.
3. 두 값의 차이가 머신 임실론(Machine Epsilon)이나 허용 오차 범위 내인지 확인한다.
4. 부동소수점은 오차가 발생하지 않으므로 == 연산자를 그대로 신뢰해도 무방하다.

8. 다음 코드에서 종괄호 초기화(Uniform Initialization)를 사용할 때 데이터 손실이 발생하는 축소 변환(narrowing conversion)을 막기 위해 컴파일 오류를 내는 코드는 무엇인가? (중)

1. int x = 3.14;
2. int x(3.14);
3. int x{3.14};
4. int x = int(3.14);

9. 다음 포인터 변수와 const의 조합 중, 포인터 변수 자신이 상수가 되어 가리키는 메모리 주소를 절대 변경할 수 없는 올바른 선언은 무엇인가? (중)

1. const int* p
2. int const* p
3. int* const p
4. const* int p

10. 상속 관계가 있는 계층 간의 안전한 다운캐스팅이나 일반적인 논리적 형 변환을 수행할 때 C 스타일 캐스팅을 대체하여 권장되는 C++의 기본 캐스팅 연산자는 무엇인가? (중)

1. reinterpret_cast
2. dynamic_cast
3. static_cast
4. const_cast

11. C++11의 범위가 있는 열거형(enum class)이 기존 C 스타일 열거형(enum)에 비해 가지는 장점이 아닌 것은 무엇인가? (중)

1. 열거형의 이름이 별도의 이름 공간 역할을 하여 심볼 이름 충돌을 방지한다.
2. 열거값이 정수형으로 암시적으로 자동 변환되는 것을 막아 타입 안전성을 높인다.
3. 열거형이 메모리에서 차지할 기본 정수 타입의 크기를 명시적으로 지정할 수 있다.

4. 선언 시 내부 멤버 변수에 초기값을 할당하지 않으면 무조건 1부터 자동 시작된다.

12. C++11에서 도입된 **auto**를 활용한 타입 추론 시, 컴파일러가 기본적으로 무시하고 버리는(제외하는) 타입 속성들의 조합으로 올바른 것은 무엇인가? (중)

1. 포인터(*)와 배열의 크기 속성
2. 참조(&)와 최상위(top-level) **const** 속성
3. 포인터(*)와 최상위 **const** 속성
4. 참조(&)와 함수의 반환 타입 속성

13. 표현식의 정확한 타입을 추론하는 기능으로, **auto**와 달리 참조(&)나 **const** 속성을 그대로 보존하여 추론하는 키워드는 무엇인가? (중)

1. typeid
2. typeof
3. constexpr
4. decltype

14. C++의 함수 오버로딩(Function Overloading)을 수행할 때, 컴파일러가 함수를 구분하는 식별 기준에 포함되지 않는 것은 무엇인가? (하)

1. 매개변수의 개수
2. 매개변수의 타입
3. 매개변수의 전달 순서
4. 함수의 반환 타입

15. C++에서 기본 인수(Default Argument)를 사용하는 함수를 작성할 때 문법적으로 반드시 지켜야 하는 규칙은 무엇인가? (중)

1. 기본 인수는 매개변수 목록의 가장 왼쪽(앞)부터 차례대로 지정해야 한다.
2. 기본 인수는 함수의 선언부와 정의부 양쪽에 중복해서 똑같이 작성해야 한다.
3. 기본 인수는 매개변수 목록의 오른쪽(끝) 매개변수부터 연속적으로 지정해야 한다.
4. 기본 인수를 지정한 함수는 어떠한 경우에도 오버로딩을 할 수 없다.

16. C++의 참조(Reference) 기능에 대한 설명으로 틀린 것은 무엇인가? (하)

1. 참조는 변수를 선언함과 동시에 반드시 원본 대상을 가리키도록 초기화되어야 한다.
2. 한 번 초기화된 참조는 프로그램 실행 도중 다른 대상을 가리키도록 변경할 수 없다.
3. 참조는 내부적으로 포인터와 완전히 동일하게 동작하므로 nullptr로 초기화할 수 있다.
4. 상수 참조(const reference)를 사용하면 원본 데이터를 복사하지 않고 안전하게 읽기 전용으로 전달할 수 있다.

17. 함수가 값을 참조 형식(return by reference)으로 반환할 때 발생할 수 있는 가장 심각한 오류의 원인이 되는 행동은 무엇인가? (상)

1. 전역 변수의 참조를 반환하는 것
2. 클래스의 멤버 변수 참조를 반환하는 것

3. 함수 내부에서 동적으로 할당한 객체의 참조를 반환하는 것
4. 함수 내부에서 지역적으로 선언된 일반 변수의 참조를 반환하는 것

18. C++17 이후 **inline** 함수 키워드의 의미와 ODR(One Definition Rule) 원칙의 관계에 대한 설명으로 가장 적절한 것은 무엇인가? (상)

1. 함수 호출부에 코드를 복사해 넣으라는 컴파일러를 향한 절대적 명령이다.
2. 인라인 함수는 디버깅이 완전히 불가능해지므로 현대 C++에서는 사용을 절대 피해야 한다.
3. 여러 번역 단위(cpp 파일)에서 동일하게 정의되는 예외를 허용하여 헤더 파일에 함수 구현을 직접 포함할 수 있게 한다.
4. 클래스 내부에 정의된 멤버 함수는 **inline** 키워드를 붙여야만 인라인 함수로 취급된다.

19. 다음 코드에서 빈칸 (A)에 들어가야 할 올바른 키워드는 무엇인가? (중)

```
void PrintValue(const ( A )& val) {
    // val을 수정할 수 없는 상수 참조 매개변수
}
```

1. 포인터 타입
2. 템플릿 타입 매개변수나 일반 타입명 (int, std::string 등)
3. auto
4. reference

20. 다음 코드 중 참조자(Reference)를 선언하는 문법으로 올바른 것을 모두 고른 것은? (하)

```
int x = 10;
int& ref1 = x;           // (A)
const int& ref2 = x;     // (B)
int & ref3 = x;          // (C)
```

1. (A)만 올바르다.
2. (A), (B)만 올바르다.
3. (B), (C)만 올바르다.
4. (A), (B), (C) 모두 올바르다.

21. 다음 코드에서 함수 오버로딩 모호성(Ambiguity) 에러를 유발하는 호출 문장은 무엇인가? (상)

```
void Func(int a, double b = 1.0);
void Func(int a);

// 호출부
```

```
Func(10);
```

1. 정상적으로 첫 번째 함수가 호출된다.
2. 정상적으로 두 번째 함수가 호출된다.
3. 기본 인수로 인해 어떤 함수를 호출해야 할지 모호하므로 컴파일 오류가 발생한다.
4. 런타임에 예외가 발생한다.

22. 다음 코드의 빈칸 (A)에 들어가야 할 올바른 캐스팅 연산자는 무엇인가? (중)

```
double pi = 3.14159;  
int intPi = ( A )<int>(pi); // 명시적이고 안전한 타입 변환
```

1. dynamic_cast
2. static_cast
3. reinterpret_cast
4. const_cast

23. 다음 중 const 포인터 선언과 의미가 잘못 연결된 것은? (중)

1. const int* p: 포인터가 가리키는 대상의 값이 상수이다.
2. int const* p: 포인터가 가리키는 대상의 값이 상수이다.
3. int* const p: 포인터 변수 자체가 상수이다.
4. const int* const p: 포인터가 가리키는 대상의 값도 변경할 수 있고, 포인터 주소도 변경할 수 있다.

24. C++11에서 도입된 auto 키워드를 사용하여 변수를 선언할 때 반드시 지켜야 하는 문법적 제약 사항은 무엇인가? (하)

1. auto 변수는 선언과 동시에 반드시 초기화되어야 한다.
2. auto 변수는 전역 변수로만 선언할 수 있다.
3. auto 변수는 함수 내부의 매개변수 위치에만 사용할 수 있다.
4. auto 변수는 상수로만 취급되므로 값을 변경할 수 없다.

25. 다음 코드에서 decltype를 이용한 타입 추론 결과로 올바른 것은? (상)

```
int x = 10;  
const int& ref = x;  
decltype(ref) y = x;
```

1. int 타입
2. const int 타입
3. const int& 타입

4. int& 타입